Chapter 1

The Actor Model and Akka

1.1 Foundations of Concurrent and Distributed Computing

1.1.1 The Concurrency Crisis and the Shared-Memory Paradigm

Modern computing architectures are fundamentally parallel. As physical constraints—primarily thermal dissipation limits and power density ceilings (the "power wall")—halted the scaling of single-core clock frequencies, hardware manufacturers transitioned to multi-core, many-core, and distributed architectures. Consequently, achieving high-throughput and low-latency software execution necessitates concurrent execution models.

Historically, the predominant abstraction for concurrent programming on shared-memory platforms has been multi-threading coupled with explicit synchronization primitives (such as mutual exclusion locks, semaphores, monitors, and atomic memory operations). Under this model, concurrent threads share a unified address space, accessing and mutating shared variables directly. However, this approach exhibits severe mathematical and engineering pathologies:

1. **Non-Deterministic Interleaving and Race Conditions**: The execution order of concurrent memory access is governed by the underlying operating system thread scheduler and hardware memory models (e.g., store buffers, out-of-order execution, and compiler optimizations). If two threads access a shared memory location concurrently without adequate synchronization, and at least one access is a write, a data race occurs. Detecting, reproducing, and debugging such race conditions is notoriously difficult due to non-deterministic scheduling.

2. **Deadlocks, Livelocks, and Lock Contention**: Coordinating access to multiple shared resources requires strict, globally enforced lock acquisition hierarchies to prevent circular wait conditions (the Coffman conditions for deadlock). As system scale grows, maintaining global locking hierarchies becomes intractable. Fine-grained locking introduces high synchronization latency, CPU cache bouncing, and high risk of deadlocks, while coarse-grained locking severely serializes execution, collapsing concurrent throughput.

3. **Violations of Object-Oriented Encapsulation**: In standard Object-Oriented Programming (OOP), an object encapsulates state and invariants, exposing methods to manipulate that state. However, in a multi-threaded shared-memory environment, an object's boundary is entirely porous: multiple threads of execution can simultaneously execute the object's methods, mutating private fields without intrinsic boundary

isolation. Thus, object-oriented encapsulation fails to protect state invariants under concurrent execution.

4. **Lack of Spatial and Network Portability**: Shared-memory synchronization primitives rely inherently on shared physical RAM, atomic hardware instructions (e.g., Compare-And-Swap - CAS), and unified cache-coherence protocols. These abstractions cannot cross network boundaries. Consequently, code written using thread locks cannot be transitioned to a distributed cluster without an architectural rewrite.

1.1.2 The Actor Model: A Mathematical Paradigm of Message-Driven Concurrency

To overcome the intrinsic failures of shared-memory multi-threading, Carl Hewitt, Peter Bishop, and Richard Steiger proposed the **Actor Model** in 1973, subsequently formalized and extended by Gul Agha (1986). The Actor Model treats the **Actor** as the universal primitive of concurrent and distributed computation.

Joe Armstrong, the co-inventor of Erlang, encapsulated the core philosophy of actor-oriented programming:

> *"The world is parallel. If we want to write programs that behave as other objects behave in the real world, then these programs will have a concurrent structure. Use a language that was designed for writing concurrent applications, and development becomes a lot easier. Erlang programs model how we think and interact."*

Complementing this, the modern engineering ethos (articulated in the Effective Go paradigm and embraced by actor systems) dictates:

> *"Do not communicate by sharing memory; instead, share memory by communicating."*

In the formal Actor Model, an actor is an autonomous computational entity that encapsulates:

- **Private State**: The internal variables of an actor are strictly confined to its execution context. No other entity in the entire system can inspect, read, or mutate an actor's private state directly.
- **A Mailbox**: A message queue associated with the actor where incoming messages are buffered until processed.
- **A Message Handler (Behavior)**: A function that dictates how the actor reacts to incoming messages.

Formally, in response to receiving a message from its mailbox, an actor can execute a finite set of operations strictly restricted to:

$$\mathcal{A}(\text{Message}) \longrightarrow \begin{cases} 1. & \text{Create a finite set of new actors: } \{\mathcal{A}'_1, \mathcal{A}'_2, \ldots, \mathcal{A}'_k\}, \\ 2. & \text{Send a finite set of messages to known actor addresses: } \{(\text{Target}_i, \text{Msg}_i)\}_{i=1}^{m}, \\ 3. & \text{Designate the replacement behavior/state } \beta_{\text{next}} \text{ for the next message.} \end{cases} \tag{1.1}$$

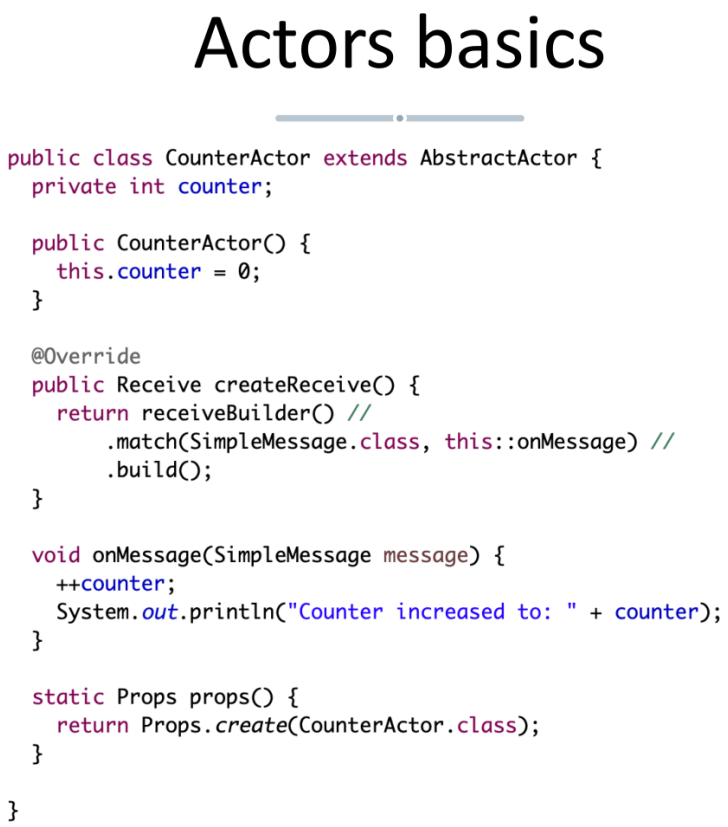


Figure 1.1: Architectural structure of an Actor: incoming messages queue into the private mailbox, processed sequentially by the message handler with complete state isolation.

1.1.3 Key Invariants and Mathematical Properties of Actors

The operational semantics of the Actor Model enforce several critical theoretical properties:

- **Thread Confinement and Absence of Shared Mutable State**: Actors never share state variables. Communication occurs exclusively via asynchronous message passing. Because an actor processes exactly one message at a time sequentially from its mailbox, the actor body is entirely free from internal data races. No synchronized blocks, mutexes, or atomic primitives are ever required within an actor's business logic.
- **Atomic Message Processing**: The processing of an individual message is atomic with respect to the actor's state: the actor processes the message from start to finish without interleaving with other messages in that actor. Incoming messages arriving concurrently are queued into the mailbox.
- **Asynchrony**: Message transmission is natively asynchronous (non-blocking). When Actor A sends a message to Actor B, Actor A does not block waiting for Actor B to receive or process the message. It resumes its own execution immediately.
- **Stateful and Persistent**: Actors maintain mutable state across sequential message invocations. Furthermore, in persistent implementations (such as Akka Persistence), an actor's state can be snapshotted or reconstructed via event sourcing from durable storage.
- **Spatial and Topological Transparency**: Actors interact using logical addresses (**Actor References**) rather than physical memory pointers or socket handles. The communication syntax is identical whether the recipient resides on the same CPU core, in the same JVM process, on a different server in the local data center, or across a global wide-area network.

1.1.4 Industrial Provenance and Adoption

The actor model is not merely a theoretical construct; it forms the technological backbone of ultra-reliable, high-scale telecommunication and enterprise distributed systems:

- **Erlang / OTP**: Developed at Ericsson by Joe Armstrong to power telecommunication switches requiring "nine nines" (99.9999999%) availability. It powers WhatsApp (enabling billions of concurrent connections with minimal hardware footprints), Apache CouchDB, and RabbitMQ.
- **Microsoft Orleans**: A virtual actor framework powering the global backend cloud infrastructure for multiplayer gaming systems such as *Halo*.
- **Akka (JVM - Java and Scala)**: Created by Jonas Bonér and Lightbend, Akka provides a high-performance actor runtime on the Java Virtual Machine. It is deployed in mission-critical distributed production systems by enterprises including BMW, Volkswagen Group, Siemens, Bosch, and Volvo.

—

1.2 The Akka Actor Architecture on the JVM

1.2.1 Core Components of the Akka Ecosystem

Akka translates the formal Actor Model into an enterprise-grade JVM framework. The runtime consists of four foundational abstractions:

1. **ActorSystem**: The heavyweight container and root context for an actor application. It allocates shared resources, including thread pools, dispatchers, configuration parameters, dead-letter mailboxes, and scheduler services. Typically, an application instantiates exactly one `ActorSystem` per JVM process:

```
ActorSystem system = ActorSystem.create("NSDS-System");

```

2. **Props**: An immutable configuration object that specifies the recipe for creating an actor instance, including the actor's implementation class, constructor parameters, and dispatcher assignment.

3. **ActorRef**: An immutable, serializable logical handle (reference) to an actor. Client code and peer actors never interact with the raw Java object instance of an actor; all interaction is strictly mediated through `ActorRef`. An `ActorRef` remains valid even if the underlying actor restarts due to a failure.

4. **AbstractActor**: The base class in Java used to implement custom actor behavior by defining the message-handling loop in the `createReceive()` method.

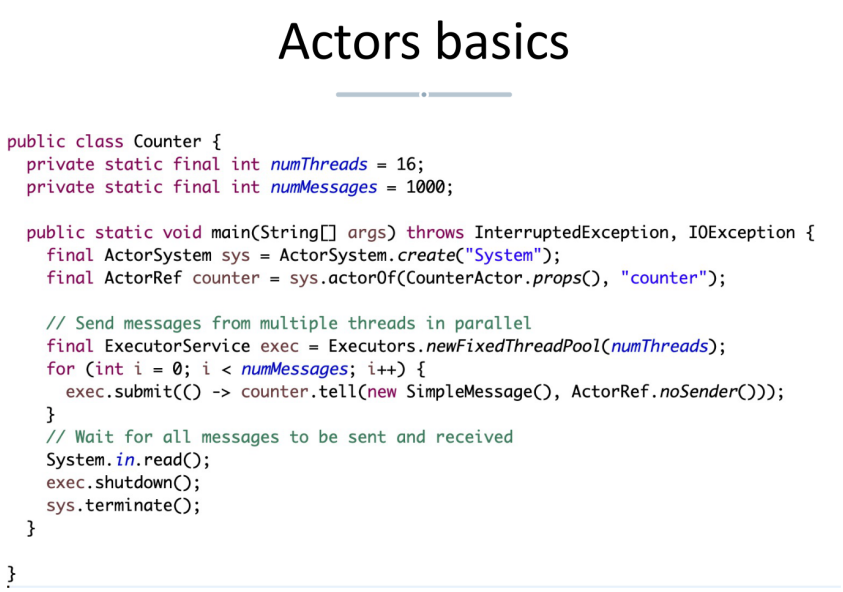


Figure 1.2: Decoupling of ActorRef, Mailbox, and Actor Instance: clients interact strictly through ActorRef, shielding the concrete actor instance from direct method invocation.

1.2.2 Creating Actors and the Static Props Pattern

To instantiate an actor, the `ActorSystem` or an existing actor context must be invoked using the `actorOf` factory method, passing a `Props` instance and an optional unique string name.

In Akka with Java, best practice dictates implementing a static factory method `props()` inside the actor class to ensure type safety and encapsulate constructor arguments:

```
public class CounterActor extends AbstractActor {
    private int counter;

    public CounterActor(int initialValue) {
        this.counter = initialValue;
    }

    // Static Props factory method
    public static Props props(int initialValue) {
        return Props.create(CounterActor.class, () -> new CounterActor(
    initialValue));
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(IncrementMessage.class, msg -> {
                this.counter += msg.getDelta();
            })
            .build();
    }
}
```

Actors are instantiated in client code as:

```
ActorSystem system = ActorSystem.create("ClusterSystem");
ActorRef counterRef = system.actorOf(CounterActor.props(0), "counterActor")
    ;
```

1.2.3 Actor Lifecycle and Lifecycle Hooks

An actor in Akka progresses through distinct lifecycle phases: **Starting**, **Receiving / Executing**, **Restarting** (upon failure), and **Stopped**. The framework provides explicit lifecycle callbacks that can be overridden to manage resources, open/close network sockets, or initialize internal state:

1.2.4 Mailbox Semantics and the Immutability Requirement

Each actor is assigned a **Mailbox** upon creation. The default mailbox implementation is unbounded and enforces **FIFO (First-In, First-Out)** queuing discipline per sender-receiver pair.

Ordering Guarantees

In Akka, message ordering satisfies the following formal guarantees:

1. **Pairwise FIFO Ordering**: If Actor A sends message M_1 and subsequently message M_2 to Actor B, then M_1 will always be delivered to Actor B's mailbox and processed before M_2, provided direct local dispatch or reliable point-to-point network channels (e.g., TCP) are used:

$$\text{Send}_A(M_1 \prec M_2) \implies \text{Receive}_B(M_1 \prec M_2) \tag{1.2}$$

Lifecycle Method	Invocation Trigger	Operational Semantic & Typical Usage
`preStart()`	Immediately after actor creation, before the first message is processed.	Initialization of private state, establishing external socket/database connections, scheduling recurring timer messages.
`preRestart(reason, message)`	Invoked on the failing actor instance right before it is terminated during a restart directive.	Disposes of allocated external resources; by default terminates all child actors and calls `postStop()`.
`postRestart(reason)`	Invoked on the newly instantiated actor instance immediately after restart.	Re-initializes state for the new instance; by default calls `preStart()`.
`postStop()`	Invoked immediately after the actor has been completely stopped and removed from the hierarchy.	Final cleanup, releasing OS handles, notifying remote monitors, flushing buffers to disk.

Table 1.1: Akka Actor Lifecycle Callback Hooks.

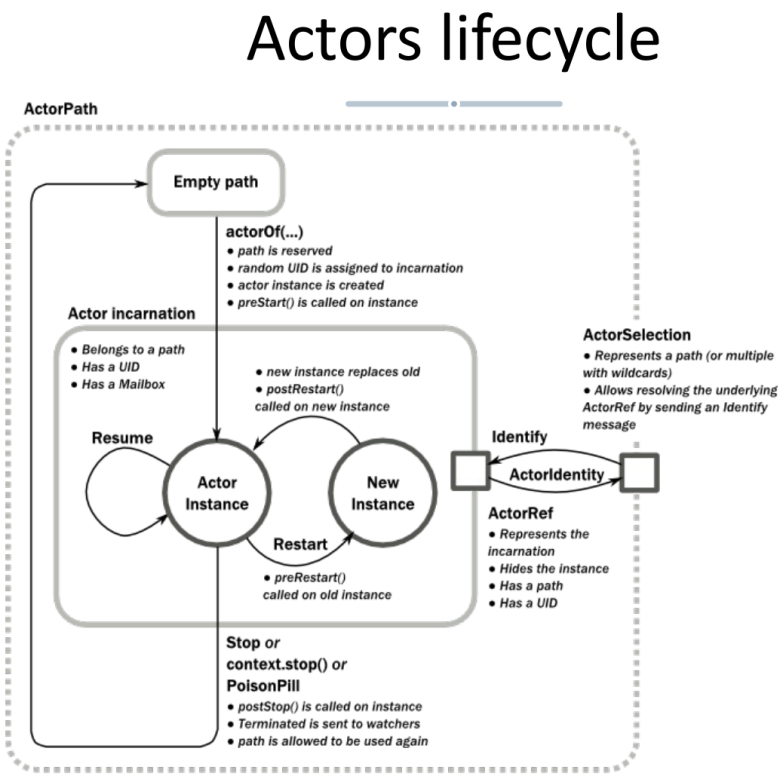


Figure 1.3: State machine depicting the complete lifecycle transitions of an Akka actor instance, highlighting restart and termination loops.

2. **Non-Deterministic Cross-Sender Interleaving**: If Actor A sends message M_A to Actor C, and Actor B sends message M_B to Actor C, there is no guaranteed relative ordering between M_A and M_B. The arrival order depends entirely on network transit latency and thread scheduling non-determinism.

The Imperative of Immutable Messages

Because Akka actors executing on the same JVM share a heap memory space, message references passed between actors on the same machine are transmitted as direct memory pointers for maximum performance (avoiding serialization overhead). Consequently:

All messages exchanged between actors MUST be strictly immutable.

If a message object contains mutable fields (e.g., public setters, exposed arrays, or mutable collections), two actors executing on separate dispatcher threads could concurrently read and mutate the message payload, resurrecting the very data races, memory visibility bugs, and cache inconsistencies that the Actor Model mathematically eliminates.

Java does not statically enforce immutability at compile time. Therefore, developers must enforce the following pattern:

```
// Canonical Immutable Message Pattern
public final class TemperatureReadingMsg implements Serializable {
    private final String sensorId;
    private final double temperature;
    private final long timestamp;

    public TemperatureReadingMsg(String sensorId, double temperature, long
    timestamp) {
        this.sensorId = sensorId;
        this.temperature = temperature;
        this.timestamp = timestamp;
    }

    public String getSensorId() { return sensorId; }
    public double getTemperature() { return temperature; }
    public long getTimestamp() { return timestamp; }
}
```

Unhandled Messages and Dead Letters

In the `createReceive()` builder, match clauses are evaluated in the order in which they are declared. If an incoming message matches no clause, Akka triggers the `unhandled(Object message)` handler. The default implementation publishes the unhandled message to the `ActorSystem`'s **Dead Letters Event Stream**. Messages addressed to terminated or non-existent actors are similarly routed to Dead Letters.

—

1.3 Communication Patterns, Protocols, and Location Transparency

1.3.1 The Tell Pattern (Asynchronous Fire-and-Forget)

The primary primitive for message transmission is `tell()`:

```
targetActorRef.tell(message, senderActorRef);
```

The `tell()` operation is completely asynchronous and non-blocking: it enqueues the message into the target actor's mailbox and immediately returns control to the caller thread.

Key operational semantics:

- **Sender Identification**: When sending from within an actor, passing `self()` as the second argument enables the receiver to reply using `sender().tell(replyMsg, self())`.
- **No Sender**: When sending from outside an actor (e.g., from a static `main` method), passing `ActorRef.noSender()` indicates that no response is expected:

```
counterActor.tell(new IncrementMessage(1), ActorRef.noSender());

```

- **Message Forwarding**: An actor can forward a message to a third-party actor while preserving the original sender reference using `forward()`:

```
workerActor.forward(message, getContext());

```

1.3.2 The Ask Pattern (Asynchronous Request-Reply with Futures)

While `tell()` represents the pure actor paradigm, software often requires interacting with non-actor boundaries (e.g., handling incoming HTTP requests, orchestrating synchronous integration tests, or querying an actor from the `main()` thread). For these scenarios, Akka provides the **Ask Pattern** via `akka.pattern.Patterns.ask`:

```
import static akka.pattern.Patterns.ask;
import scala.concurrent.Future;
import scala.concurrent.Await;
import scala.concurrent.duration.Duration;
import java.util.concurrent.TimeUnit;

Duration timeout = Duration.create(3, TimeUnit.SECONDS);
Future<Object> future = ask(targetActorRef, new QueryBalanceMsg(accountId),
    3000);

// Synchronously blocking the calling thread until completion or timeout
BalanceResponse reply = (BalanceResponse) Await.result(future, timeout);
```

Internal Mechanics and Performance Overhead of Ask

Under the hood, the Ask Pattern does not alter the recipient actor's execution:

1. Akka instantiates an ephemeral, single-use **Promise Actor** within the `ActorSystem`.
2. The message is sent to the target actor with the Promise Actor's reference passed as the implicit `sender()`.
3. A timer is registered with the system scheduler for the specified timeout duration.
4. When the target actor replies via `sender().tell(response, self())`, the Promise Actor receives the reply, fulfills the underlying Scala/Java `Future`, and immediately terminates.

5. If the timeout expires before a reply arrives, the Future fails with an `AskTimeoutException`.

Dimension	The Tell Pattern (`tell`)	The Ask Pattern (`ask`)
Execution Model	Purely asynchronous, non-blocking fire-and-forget.	Asynchronous returning a Future; can be blocked via `Await.result()`.
Resource Overhead	Extremely lightweight: enqueues directly into recipient's mailbox.	Medium: allocates a temporary Promise Actor, a timer, and a Future handle.
Failure Handling	Transparent: network drop or crash handled via supervisor or dead letters.	Explicit: fails with `AskTimeoutException` if target crashes or drops message.
Best Used For	Internal actor-to-actor communication, high-throughput pipelines.	Bridging non-actor code to actors (e.g., REST endpoints, `main` tests).

Table 1.2: Comparative Analysis: Tell Pattern vs. Ask Pattern.

1.3.3 Delivery Guarantees: Why Best-Effort (At-Most-Once)?

Distributed systems theory establishes that in the presence of unreliable network channels, guaranteeing exactly-once delivery without coordination overhead is impossible (the Two Generals' Problem).

Akka adopts a strict **at-most-once (best-effort)** delivery semantic by default:

- Messages may be lost if an actor crashes, if a mailbox overflows, or if a network link partitions.
- Messages are never duplicated by the transport layer itself.

Architectural Justification ("Less is More"): As emphasized by Professor Mottola, providing at-least-once or guaranteed delivery at the transport layer imposes severe costs: sequence numbering, acknowledgment packets, retransmission buffers, state persistence, and deduplication caches. In distributed environments:

1. The underlying Internet Protocol (IP) is natively best-effort.
2. TCP provides reliable byte-streams over single connections, but cannot mask machine crashes, network partitions, or process restarts.
3. Many domain operations are naturally idempotent or time-sensitive (e.g., periodic sensor telemetry, where a dropped packet is superseded by the next reading).
4. Applications requiring stronger guarantees can implement application-level acknowledgments and retries, or integrate durable message queues (such as Apache Kafka).

1.3.4 Location Transparency and Akka Remoting

One of the most powerful architectural capabilities of Akka is **Location Transparency**. An actor reference (`ActorRef`) encapsulates identity independently of physical topology. Whether an actor resides on the local JVM or on a remote node across a network, message invocation remains syntactically identical: `ref.tell(msg, self())`.

Actor Paths and Hierarchical Addressing

Every actor is uniquely identified by an **Actor Path**, structured analogously to a hierarchical filesystem URI:

`akka.<protocol>://<actor-system-name>@<hostname>:<port>/user/<actor-hierarchy-path>` (1.3)

For example:

- Local path: `akka://NSDS-System/user/supervisor/worker-1`
- Remote TCP path: `akka.tcp://ClusterNode@192.168.1.100:2552/user/dataProcessor`

Configuring Remote Nodes via HOCON

Akka uses the Human-Optimized Config Object Notation (HOCON) for configuration (typically in `application.conf`):

```
akka {
  actor {
    provider = remote
  }
  remote {
    enabled-transports = ["akka.remote.netty.tcp"]
    netty.tcp {
      hostname = "127.0.0.1"
      port = 6123
    }
  }
}
```

Resolving Remote Actors: ActorSelection

To communicate with a remote actor whose `ActorRef` is not yet locally available, a client program resolves the remote actor's logical URI using `actorSelection`:

```
// Client lookup of remote server actor
String remoteAddress = "akka.tcp://ServerSystem@127.0.0.1:6123/user/
    serverActor";
ActorSelection remoteSelection = getContext().actorSelection(remoteAddress)
    ;

// Messages can be sent directly to an ActorSelection
remoteSelection.tell(new QueryMessage("data"), getSelf());
```

Once the remote actor replies, the local actor acquires the remote actor's concrete `ActorRef` via `sender()`, enabling direct, cached communication without subsequent URI resolutions.

—

1.4 Dynamic Behavior Switching and the Stash Mechanism

1.4.1 Finite State Machines via the Become/Unbecome Pattern

In many concurrent protocols, an actor must fundamentally alter how it reacts to incoming messages depending on its operational state (e.g., transitioning from *Disconnected* to *Connecting*, *Active*, *Busy*, or *Suspended*).

Rather than cluttering message handling logic with complex, error-prone conditional branches (`if-else` or `switch` statements inspecting internal status flags), Akka empowers actors to dynamically swap their entire message-handling function at runtime using `getContext().become()`:

```
public class ProtocolActor extends AbstractActor {

    @Override
    public Receive createReceive() {
        return disconnectedBehavior();
    }

    private Receive disconnectedBehavior() {
        return receiveBuilder()
            .match(ConnectMsg.class, msg -> {
                // Perform connection logic...
                System.out.println("Connecting to " + msg.getHost());
                getContext().become(connectedBehavior());
            })
            .match(DataMsg.class, msg -> {
                System.err.println("Rejected data: Not connected!");
            })
            .build();
    }

    private Receive connectedBehavior() {
        return receiveBuilder()
            .match(DataMsg.class, msg -> {
                System.out.println("Processing data: " + msg.getPayload());
            })
            .match(DisconnectMsg.class, msg -> {
                System.out.println("Disconnecting...");
                getContext().become(disconnectedBehavior());
            })
            .build();
    }
}
```

The `become()` method accepts a second boolean parameter, `discardOld` (defaulting to `true`). If set to `false`, Akka pushes the new behavior onto an internal behavior stack. The actor can subsequently revert to its previous behavior by invoking `getContext().unbecome()`.

1.4.2 The Stash Mechanism: AbstractActorWithStash

Often, an actor in a particular operational state cannot immediately process certain incoming messages, yet discarding them would violate application protocol requirements. The **Stash** pattern enables an actor to set aside messages temporarily and replay them later when its state permits.

To utilize stashing, an actor must:

1. Inherit from `AbstractActorWithStash` (or implement the `Stash` trait).
2. Invoke `stash()` on messages that must be deferred. This removes the current message from the active dispatch loop and appends it to a private, secondary stash buffer.
3. When transitioning to an operational state capable of handling deferred messages, invoke `unstashAll()`.

Internal Mailbox Repositioning Semantics

When `unstashAll()` is executed, Akka transfers all messages currently in the stash buffer back into the actor's primary mailbox. Crucially:

> **Stashed messages are prepended to the front of the primary mailbox in the exact order in which they were stashed.**

This guarantees that stashed messages are processed immediately upon behavior transition, ahead of newly arriving messages, preserving local causal progression.

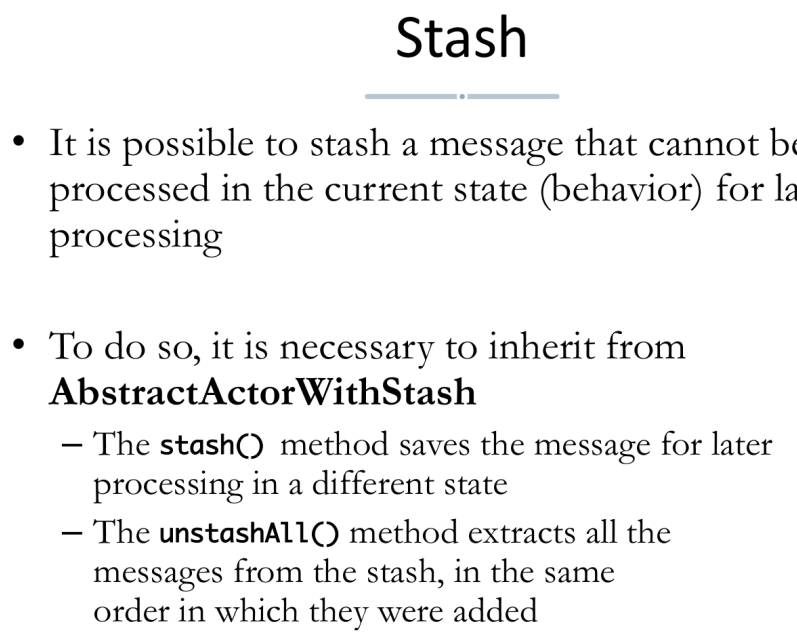


Figure 1.4: The Stash buffer mechanics: deferred messages are stashed and later prepended to the primary mailbox upon unstashAll().

—

1.5 Fault Tolerance and the Supervision Hierarchy

1.5.1 The "Let It Crash" Philosophy

In traditional monolithic programming (e.g., standard Java), error handling is tightly coupled with business logic through defensive `try-catch` blocks. When an unhandled exception propagates up the call stack, threads crash unpredictably, frequently leaving shared data structures corrupted or locks permanently held.

The Actor Model adopts Joe Armstrong's radical **"Let It Crash"** philosophy:

- **Decoupling Error Handling from Business Logic**: Developers do not write defensive code attempting to handle catastrophic failures inside the core message processing loop. If a fatal runtime anomaly occurs (e.g., database connection dropped, corrupted payload, null pointer), the actor simply throws an exception and crashes.
- **Supervision from Outside**: Fault management is completely externalized. When an actor crashes, its parent in the actor hierarchy—its **Supervisor**—is automatically notified by the Akka runtime and makes an architectural decision on how to recover.

1.5.2 The Actor Hierarchy

In Akka, actors do not exist in isolation; they are organized into a strict tree hierarchy. Every actor created via `getContext().actorOf()` is a child of the actor that created it.

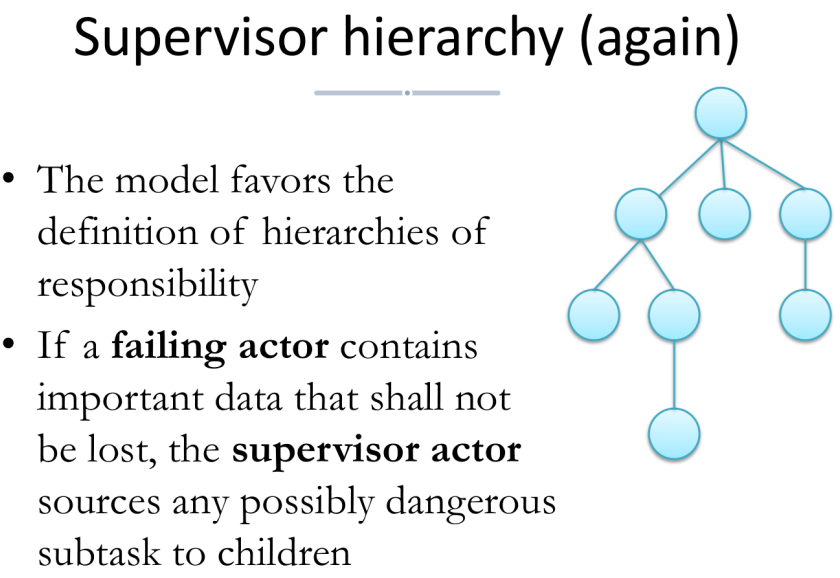


Figure 1.5: The Akka Supervision Tree Hierarchy: showing Root Guardian, User Guardian (/user), and nested application supervisor/worker topologies.

The hierarchy consists of three structural tiers:

1. **Root Guardian (/)**: The top of the hierarchy, supervising the internal system guardians.
2. **System Guardian (`/system`)**: Manages internal framework services, logging actors, remoting endpoints, and dead letters.
3. **User Guardian (`/user`)**: The parent of all top-level application actors created via `system.actorOf()`. Every user-defined actor has a supervisor.

1.5.3 Supervision Strategies and Directives

A supervisor defines its recovery policy by overriding the `supervisorStrategy()` method, returning an instance of either `OneForOneStrategy` or `AllForOneStrategy`:

- **OneForOneStrategy**: The recovery directive chosen by the supervisor is applied **only** to the specific child actor that crashed. Peer child actors continue operating without disruption. This is the optimal strategy when children perform independent workloads.
- **AllForOneStrategy**: The recovery directive chosen by the supervisor is applied to **all** child actors belonging to that supervisor, even if only one child failed. This strategy is mandatory when child actors participate in a tightly coupled, cooperative transaction where the failure of one invalidates the collective state of the group.

The Four Supervisory Directives

Upon intercepting a child failure, the supervisor executes one of four formal directives:

1. **Resume**: The crashed actor instance is **not** terminated. It retains its entire private in-memory state and immediately proceeds to process the next message in its mailbox.
    - *Use Case*: Minor, transient, or non-state-corrupting anomalies (e.g., a temporary network timeout, a malformed sensor packet that can be discarded without invalidating historical aggregates).
2. **Restart** (*Default Policy*): The failing actor instance is destroyed and a brand-new instance of the actor class is instantiated via its original `Props`.
    - *State Semantic*: The actor's private state is completely reset to its initial condition.
    - *Mailbox Semantic*: The actor's mailbox is **preserved**. Unprocessed messages remain queued, and the new instance processes them sequentially.
    - *Lifecycle Hook Invocations*: The old instance executes `preRestart(reason, message)`; the new instance executes `postRestart(reason)`.
    - *Use Case*: State corruption or unrecoverable local exceptions where re-initializing from a clean slate restores operational integrity.
3. **Stop**: The failing actor instance is permanently terminated. Its mailbox is drained (unprocessed messages are redirected to Dead Letters), all its child actors are recursively terminated, and its `ActorRef` becomes defunct.
    - *Use Case*: Unrecoverable failures, critical resource exhaustion, or terminal operational errors.
4. **Escalate**: The supervisor confesses inability to handle the exception, escalating the failure up the hierarchy to its own parent supervisor. The supervisor itself suspends its own operation and all its children while waiting for its parent's directive.

Defining a Supervision Strategy in Code

The following complete Java implementation demonstrates configuring a `OneForOneStrategy` with retry limits and time windows:

```
1  import akka.actor.AbstractActor;
2  import akka.actor.OneForOneStrategy;
3  import akka.actor.SupervisorStrategy;
4  import akka.japi.pf.DeciderBuilder;
5  import java.time.Duration;
6
7  public class CustomSupervisorActor extends AbstractActor {
8
9      // Strategy: maximum 10 retries within a 1-minute moving window
10     private static final SupervisorStrategy STRATEGY = new
       OneForOneStrategy(
11         10,
12         Duration.ofMinutes(1),
13         DeciderBuilder
14             .match(IllegalArgumentException.class, e -> SupervisorStrategy.
       resume())
```

```
15              .match(NullPointerException.class, e -> SupervisorStrategy.
    restart())
16              .match(IllegalStateException.class, e -> SupervisorStrategy.
    stop())
17              .matchAny(e -> SupervisorStrategy.escalate())
18              .build()
19      );
20
21      @Override
22      public SupervisorStrategy supervisorStrategy() {
23          return STRATEGY;
24      }
25
26      @Override
27      public Receive createReceive() {
28          return receiveBuilder()
29              .match(Props.class, childProps -> {
30                  // Spawning a supervised child actor
31                  ActorRef child = getContext().actorOf(childProps);
32                  sender().tell(child, self());
33              })
34              .build();
35      }
36  }
```

—

1.6 Akka Clustering and Distributed Topology

1.6.1 Decentralized Cluster Membership Architecture

Akka Cluster extends location transparency across a dynamic pool of networked nodes without relying on centralized coordination masters (such as Apache ZooKeeper). It provides:

- **Decentralized Peer-to-Peer Topology**: All nodes are symmetric members; there is no single point of failure (SPOF) or central bottleneck.
- **Gossip Protocol**: Cluster membership state is disseminated asynchronously through periodic, randomized pairwise state gossip.
- **Failure Detection**: Automatic, distributed monitoring of node reachability.

1.6.2 The Gossip Protocol and Vector Clock Convergence

Nodes organize into an overlay network. At regular intervals (default: 1 second), each node randomly selects another cluster member and exchanges membership views.

- **Vector Clocks for Causal History**: Each gossip message includes a **Vector Clock** tracking the partial order of membership state modifications observed across nodes (e.g., nodes joining, leaving, or becoming unreachable).
- **Gossip Convergence**: A node attains cluster state convergence when it can mathematically verify that all reachable cluster members share an identical view of the cluster membership.

- **The Convergence Invariant**: Cluster topology changes (such as promoting a node from *Joining* to *Up*) can only be executed by the designated **Cluster Leader** once full gossip convergence is achieved across all reachable members.

1.6.3 Node Lifecycle and Failure Detection

A node in an Akka Cluster progresses through formal lifecycle states:

$$\text{Joining} \longrightarrow \text{Up} \longrightarrow \text{Leaving} \longrightarrow \text{Exiting} \longrightarrow \text{Down} \longrightarrow \text{Removed} \tag{1.4}$$

The Φ Accrual Failure Detector

Akka implements the Hayashibara et al. Φ **Accrual Failure Detector**. Rather than making binary (up/down) decisions based on fixed heartbeat timeouts, the detector continuously measures heartbeat inter-arrival intervals, fitting them to a normal distribution. It outputs a continuous suspicion metric Φ:

$$\Phi = -\log_{10}(P_{\text{later}}(t - t_{\text{last}})) \tag{1.5}$$

When network latency spikes due to transient traffic congestion, Φ rises gracefully. If Φ exceeds a configured threshold (default: 10), the node is marked **Unreachable**. Unreachable nodes prevent gossip convergence until they recover or are explicitly marked **Down**.

1.6.4 Seed Nodes and Higher-Level Cluster Tools

- **Seed Nodes**: Configured contact addresses serving as bootstrap rendezvous points for joining nodes. A new node contacts all configured seed nodes, issuing a join command to the first that responds.
- **Cluster Singleton**: Guarantees that exactly one actor of a given type executes across the entire cluster, with automatic failover if the hosting node crashes.
- **Cluster Sharding**: Automatically partitions a large collection of stateful actors across all cluster nodes, routing messages to their current location via entity identifiers.
- **Distributed Data**: Replicates state across nodes using Conflict-Free Replicated Data Types (CRDTs), ensuring eventual consistency without locks.

—

1.7 System Analysis, Complexity, and Architectural Trade-offs

1.7.1 Complexity and Resource Footprint Analysis

1.7.2 Comparative Architectural Trade-offs

When designing networked systems with Akka, developers navigate crucial trade-offs:

1. **Thread Pool Dispatcher Tuning**: Akka executes actors using an underlying ForkJoin or ThreadPool executor. Long-running, blocking operations (e.g., blocking JDBC queries or legacy synchronous HTTP calls) executed inside an actor will starve the dispatcher's shared worker threads, halting unrelated actors. *Mitigation*: Offload blocking operations to a dedicated, isolated thread pool using Futures.

Metric / Dimension	Quantitative Bound	Architectural Analysis & Trade-off
Memory Footprint	$\approx 300 - 500$ bytes per Actor instance.	In contrast to an OS thread ($\approx$ 1 MB stack), millions of actors can run concurrently on standard JVM heap memory.
Message Transmission Complexity	$O(1)$ point-to-point; $O(N)$ multicast.	Point-to-point dispatch is direct memory pointer passing locally, or single socket write over TCP.
Gossip Convergence Latency	$O(\log N)$ rounds in an N-node cluster.	Dissemination scales logarithmically with cluster size; heartbeat overhead is bounded.
Concurrency Bottlenecks	Zero lock contention within an actor.	Bottlenecks occur only if a single actor's mailbox receives messages faster than its sequential processing rate.
Fault Recovery Cost	Restart: $O(1)$ object reallocation.	Restarting an actor preserves the queued mailbox, minimizing data loss during local transient recovery.

Table 1.3: Quantitative Performance and Resource Characteristics of Akka.

2. **Mailbox Sizing: Unbounded vs Bounded**: Unbounded mailboxes prevent message loss under traffic bursts, but risk `OutOfMemoryError` if consumption lags behind arrival indefinitely. Bounded mailboxes enforce capacity limits, dropping messages or rejecting senders when full.
3. **Supervision Granularity**: Fine-grained hierarchies localize failure blast radiuses (one failing child does not degrade peers), but introduce slight tree traversal overhead.

—

1.8 Review Questions and Exam / Evaluation Lab Problem Solutions

1.8.1 Conceptual Review Questions

Question 1: Explain the operational differences between the Resume and Restart supervisory directives in Akka. Under what circumstances must an engineer select Resume instead of Restart? Solution: When a supervisor executes the `Resume` directive upon an actor exception, the current instance of the actor is retained in memory. Its private state variables remain entirely intact, and the actor immediately processes the next message queued in its mailbox.

In contrast, when the supervisor executes the `Restart` directive:

1. The crashed actor instance is disposed of; its `preRestart(reason, message)` hook is executed.
2. A new instance of the actor is created via its original `Props`, executing `postRestart(reason)`.
3. The actor's private state is reset to initial constructor values.
4. The mailbox is preserved; queued messages are processed by the newly instantiated actor.

Selection Criteria: An engineer must select `Resume` when the encountered exception is transient, external, or non-corrupting (e.g., a malformed telemetry reading, a temporary network socket read timeout, or an unparseable payload). In such scenarios, the actor's historical state (e.g., accumulated running averages, active client subscriptions, or cached session tokens) is perfectly valid and must not be destroyed.

Question 2: Why must messages exchanged in Akka be strictly immutable, and how does the Ask Pattern differ internally from the Tell Pattern? Solution: In Akka, actors running on the same JVM share a common memory heap. To achieve ultra-high throughput, the framework passes message object references directly without deep serialization. If messages were mutable, concurrent threads executing separate actors could read and mutate the same object fields concurrently, causing data races, torn reads, and cache inconsistency.

The `tell()` operation is pure fire-and-forget: it enqueues the message into the recipient's mailbox and immediately yields execution, incurring minimal CPU overhead. The `ask()` pattern creates an ephemeral Promise Actor, registers a timeout handler with the system scheduler, sends the message with the Promise Actor as the sender, and returns a `Future`. The Ask pattern introduces allocation and scheduling overhead and can lead to thread starvation if synchronously blocked with `Await.result()`.

—

1.8.2 Evaluation Lab 2022 Solution: Partitioned Pub/Sub Broker with Dynamic Batching

Problem Specification

Design a distributed Publish-Subscribe system in Akka comprising:

- **Two Worker Actors**: Match published events against active subscriptions.
- **A Broker Actor**: Coordinates workers, partitioning subscriptions based on key parity (even keys to Worker 1, odd keys to Worker 2).
- **Fault Tolerance**: If a worker receives a `PublishMsg` for an unknown topic, it raises an exception. The broker must supervise workers using a strategy that preserves all existing subscriptions (`Resume`).
- **Dynamic Batching**: When receiving `BatchMsg(isOn = true)`, the broker must buffer subsequent events. Upon receiving `BatchMsg(isOn = false)`, all buffered events must be processed in batch.

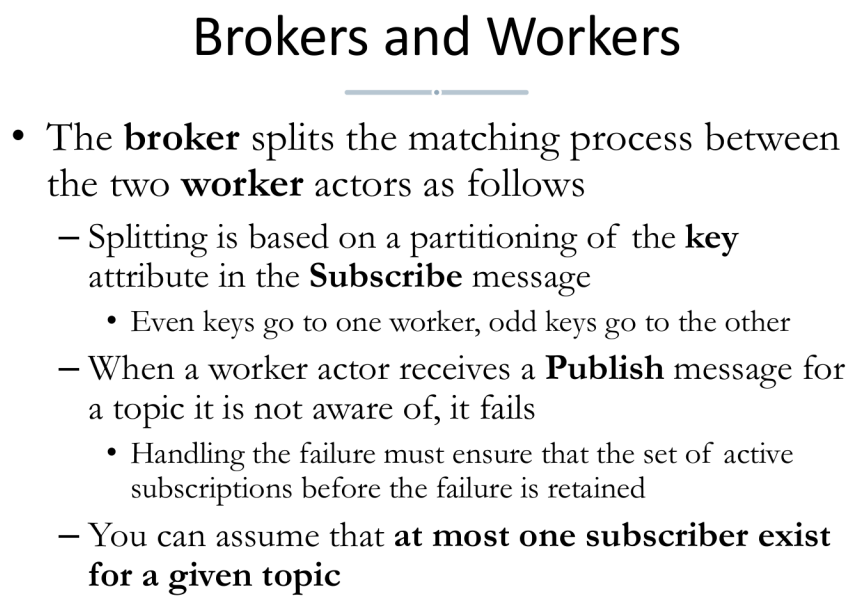


Figure 1.6: Evaluation Lab 2022 Architecture: Broker partitioning subscriptions across even/odd workers, handling dynamic batching and supervisor recovery.

Complete Implementation

```
import akka.actor.ActorRef;
import java.io.Serializable;

public class Messages {
    public static class SubscribeMsg implements Serializable {
        private final String topic;
        private final int key;
        private final ActorRef subscriber;

        public SubscribeMsg(String topic, int key, ActorRef subscriber) {
```

```
            this.topic = topic;
            this.key = key;
            this.subscriber = subscriber;
        }
        public String getTopic() { return topic; }
        public int getKey() { return key; }
        public ActorRef getSubscriber() { return subscriber; }
    }

    public static class PublishMsg implements Serializable {
        private final String topic;
        private final int key;
        private final String payload;

        public PublishMsg(String topic, int key, String payload) {
            this.topic = topic;
            this.key = key;
            this.payload = payload;
        }
        public String getTopic() { return topic; }
        public int getKey() { return key; }
        public String getPayload() { return payload; }
    }

    public static class NotifyMsg implements Serializable {
        private final String payload;
        public NotifyMsg(String payload) { this.payload = payload; }
        public String getPayload() { return payload; }
    }

    public static class BatchMsg implements Serializable {
        private final boolean on;
        public BatchMsg(boolean on) { this.on = on; }
        public boolean isOn() { return on; }
    }
}
```

Listing 1.1: Message Definitions for Pub/Sub System

```
import akka.actor.AbstractActor;
import akka.actor.Props;
import java.util.HashMap;
import java.util.Map;

public class WorkerActor extends AbstractActor {
    // Subscriptions map: Topic -> Subscriber ActorRef
    private final Map<String, ActorRef> subscriptions = new HashMap<>();

    public static Props props() {
        return Props.create(WorkerActor.class, WorkerActor::new);
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(Messages.SubscribeMsg.class, msg -> {
                subscriptions.put(msg.getTopic(), msg.getSubscriber());
                System.out.println(self().path().name() + " registered
    topic: " + msg.getTopic());
            })
            .match(Messages.PublishMsg.class, msg -> {
```

```
                if (!subscriptions.containsKey(msg.getTopic())) {
                    System.err.println(self().path().name() + " fault:
    Unknown topic " + msg.getTopic());
                    throw new Exception("Unknown topic: " + msg.getTopic())
    ;
                }
                ActorRef sub = subscriptions.get(msg.getTopic());
                sub.tell(new Messages.NotifyMsg(msg.getPayload()), self());
            })
            .build();
    }
}
```

Listing 1.2: WorkerActor with Fault Injection on Unknown Topics

```
import akka.actor.*;
import akka.japi.pf.DeciderBuilder;
import java.time.Duration;
import java.util.LinkedList;
import java.util.Queue;

public class BrokerActor extends AbstractActorWithStash {
    private final ActorRef workerEven;
    private final ActorRef workerOdd;
    private final Queue<Messages.PublishMsg> batchBuffer = new LinkedList
    <>();

    // Supervision Strategy: Resume on exception to retain subscriptions
    private static final SupervisorStrategy STRATEGY = new
    OneForOneStrategy(
        10,
        Duration.ofMinutes(1),
        DeciderBuilder
            .match(Exception.class, e -> SupervisorStrategy.resume())
            .build()
    );

    @Override
    public SupervisorStrategy supervisorStrategy() {
        return STRATEGY;
    }

    public BrokerActor() {
        this.workerEven = getContext().actorOf(WorkerActor.props(), "
    workerEven");
        this.workerOdd = getContext().actorOf(WorkerActor.props(), "
    workerOdd");
    }

    public static Props props() {
        return Props.create(BrokerActor.class, BrokerActor::new);
    }

    @Override
    public Receive createReceive() {
        return normalBehavior();
    }

    // Normal Behavior: immediate routing
    private Receive normalBehavior() {
        return receiveBuilder()
```

```
            .match(Messages.BatchMsg.class, msg -> {
                if (msg.isOn()) {
                    System.out.println("Broker transitioning to BATCH mode.
");
                    getContext().become(batchBehavior());
                }
            })
            .match(Messages.SubscribeMsg.class, this::routeSubscription)
            .match(Messages.PublishMsg.class, this::routePublish)
            .build();
    }

    // Batch Behavior: buffer incoming publish events
    private Receive batchBehavior() {
        return receiveBuilder()
            .match(Messages.BatchMsg.class, msg -> {
                if (!msg.isOn()) {
                    System.out.println("Broker flushing batch (" +
batchBuffer.size() + " messages).");
                    while (!batchBuffer.isEmpty()) {
                        routePublish(batchBuffer.poll());
                    }
                    getContext().become(normalBehavior());
                }
            })
            .match(Messages.SubscribeMsg.class, this::routeSubscription)
            .match(Messages.PublishMsg.class, batchBuffer::add)
            .build();
    }

    private void routeSubscription(Messages.SubscribeMsg msg) {
        if (msg.getKey() % 2 == 0) {
            workerEven.forward(msg, getContext());
        } else {
            workerOdd.forward(msg, getContext());
        }
    }

    private void routePublish(Messages.PublishMsg msg) {
        if (msg.getKey() % 2 == 0) {
            workerEven.forward(msg, getContext());
        } else {
            workerOdd.forward(msg, getContext());
        }
    }
}
```

Listing 1.3: BrokerActor with Dynamic Batching and Resume Supervision

—

1.8.3 Evaluation Lab 2023 Solution: Sensor Data Processing with Policy Dispatching

Problem Specification

Develop an industrial temperature monitoring pipeline in Akka with:

1. **SensorDataProcessor Actors**: Maintain the running average of received tempera-

ture readings:

$$\bar{T}_k = \frac{(k-1)\bar{T}_{k-1} + T_k}{k} \tag{1.6}$$

If a negative temperature reading is received, the processor raises an exception. Fault recovery must ensure the average temperature before failure is preserved, but the negative reading is excluded (`Resume`).

2. **DispatcherActor**: Dynamically routes incoming readings according to two policies configurable via `DispatchLogicMsg`:
   - **Load Balancing**: Permanently assigns each sensor to the least-loaded data processor.
   - **Round Robin**: Rotates incoming readings sequentially across data processors.

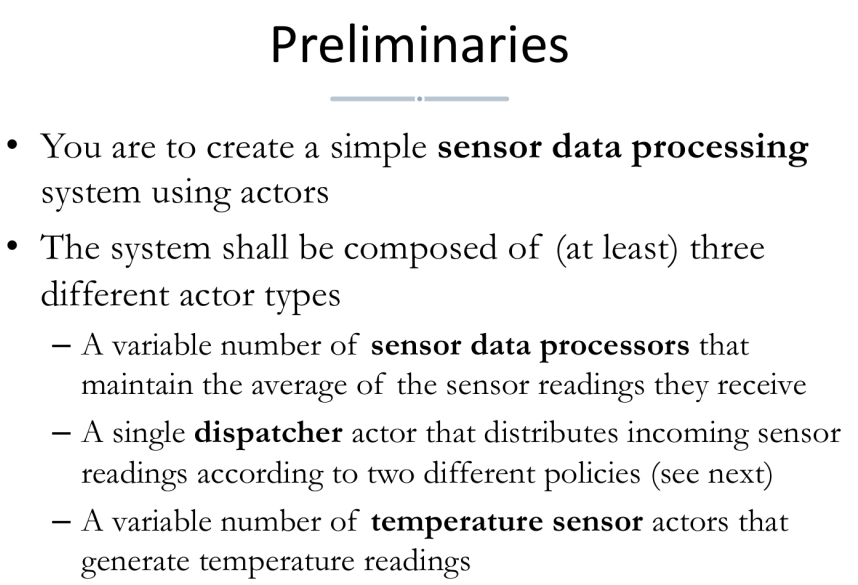


Figure 1.7: Evaluation Lab 2023 Architecture: Dispatcher routing sensor readings via dynamic Load Balancing or Round Robin to resilient Data Processors.

Complete Implementation

```
1  import akka.actor.AbstractActor;
2  import akka.actor.Props;
3
4  public class SensorProcessorActor extends AbstractActor {
5      private double sum = 0.0;
6      private int count = 0;
7
8      public static Props props() {
9          return Props.create(SensorProcessorActor.class,
       SensorProcessorActor::new);
10     }
11
12     @Override
13     public Receive createReceive() {
14         return receiveBuilder()
15             .match(TemperatureMsg.class, msg -> {
```

```
                if (msg.getValue() < 0) {
                    System.err.println(self().path().name() + " rejecting
    invalid temperature: " + msg.getValue());
                    throw new IllegalArgumentException("Negative
    temperature invalid: " + msg.getValue());
                }
                count++;
                sum += msg.getValue();
                double currentAvg = sum / count;
                System.out.printf("%s -> Read: %.2f | New Running Avg: %.2f
     (Total readings: %d)%n",
                    self().path().name(), msg.getValue(), currentAvg, count
    );
            })
            .build();
    }
}
```

Listing 1.4: SensorProcessorActor with State Invariant Preservation

```
import akka.actor.*;
import akka.japi.pf.DeciderBuilder;
import java.time.Duration;
import java.util.*;

public class DispatcherActor extends AbstractActor {
    private final List<ActorRef> processors;
    private final Map<ActorRef, ActorRef> sensorToProcessorMap = new
    HashMap<>();
    private final Map<ActorRef, Integer> processorLoadMap = new HashMap<>()
    ;
    private int roundRobinIndex = 0;
    private boolean isRoundRobin = true; // Default policy

    private static final SupervisorStrategy STRATEGY = new
    OneForOneStrategy(
        10,
        Duration.ofMinutes(1),
        DeciderBuilder
            .match(IllegalArgumentException.class, e -> SupervisorStrategy.
    resume())
            .build()
    );

    @Override
    public SupervisorStrategy supervisorStrategy() {
        return STRATEGY;
    }

    public DispatcherActor(int numProcessors) {
        this.processors = new ArrayList<>();
        for (int i = 0; i < numProcessors; i++) {
            ActorRef p = getContext().actorOf(SensorProcessorActor.props(),
     "processor-" + i);
            processors.add(p);
            processorLoadMap.put(p, 0);
        }
    }

    public static Props props(int numProcessors) {
        return Props.create(DispatcherActor.class, () -> new
```

```
    DispatcherActor(numProcessors));
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(DispatchLogicMsg.class, msg -> {
                this.isRoundRobin = msg.isRoundRobin();
                System.out.println("Dispatcher switched logic to: " +
                    (isRoundRobin ? "ROUND ROBIN" : "LOAD BALANCING"));
            })
            .match(TemperatureMsg.class, msg -> {
                ActorRef targetProcessor;
                if (isRoundRobin) {
                    targetProcessor = processors.get(roundRobinIndex);
                    roundRobinIndex = (roundRobinIndex + 1) % processors.
    size();
                } else {
                    ActorRef sensorSender = sender();
                    if (!sensorToProcessorMap.containsKey(sensorSender)) {
                        // Find processor with minimum assigned sensors
                        targetProcessor = processors.stream()
                            .min(Comparator.comparingInt(processorLoadMap::
    get))
                            .orElse(processors.get(0));
                        sensorToProcessorMap.put(sensorSender,
    targetProcessor);
                        processorLoadMap.put(targetProcessor,
    processorLoadMap.get(targetProcessor) + 1);
                    } else {
                        targetProcessor = sensorToProcessorMap.get(
    sensorSender);
                    }
                }
                targetProcessor.forward(msg, getContext());
            })
            .build();
    }
}
```

Listing 1.5: DispatcherActor with Dynamic Routing Policies and Fault Recovery